Task Manager Assessment Submission

Candidate	Shih-En Chen
Repository	https://github.com/jackychen21/pulseboard
Live Demo	https://pulseboard-red.vercel.app
Stack	React, TypeScript, Vite, Supabase, anonymous auth

1. Solution Overview

PulseBoard is a responsive four-column Kanban board that lets each guest user create tasks, move them across statuses, and visually manage work. The interface emphasizes strong hierarchy, clear spacing, fast scanability, and immediate feedback for loading, empty, and error states.

2. Design Decisions

- Used a glassmorphism-inspired dark surface with warm accent gradients to make the board feel intentional instead of generic.
- Kept drag-and-drop native with the HTML5 API to reduce dependency risk while preserving direct interaction.
- Added summary cards, due date urgency badges, and filtering so the board is more useful than a basic todo list.
- Included a demo fallback mode so the product can still be reviewed locally before Supabase credentials are added.

3. Features Implemented

- Anonymous guest session flow through Supabase Auth.
- Per-user task isolation with Row Level Security.
- Task creation with title, description, priority, and due date.
- Drag-and-drop status changes across To Do, In Progress, In Review, and Done.
- Search, priority filtering, and due date filtering.
- Summary statistics for total, completed, in-flight, overdue, and due-soon tasks.
- Realtime-ready subscription hook for task refresh when Supabase mode is enabled.

4. Database Schema

The full SQL setup is included in supabase/schema.sql. The main tasks table is summarized below.

Field	Type	Notes
id	uuid	Primary key
title	text	Required
status	text	todo, in_progress, in_review, done
description	text	Optional detail body, defaults to empty string
priority	text	low, normal, high

user_id	uuid	Tied to anonymous guest session via auth.uid()

Additional field included in the actual schema: created_at timestamptz default timezone('utc', now()).

Full SQL (supabase/schema.sql)

```
create extension if not exists "pgcrypto";

create table if not exists public.tasks (
  id uuid primary key default gen_random_uuid(),
  title text not null,
  description text not null default '',
  status text not null default 'todo' check (status in ('todo',
'in_progress', 'in_review', 'done')),
  priority text not null default 'normal' check (priority in ('low',
'normal', 'high')),
  due_date date,
  user_id uuid not null default auth.uid(),
  created_at timestamptz not null default timezone('utc', now())
);

alter table public.tasks enable row level security;

create policy "Users can read their own tasks"
on public.tasks
for select
using (auth.uid() = user_id);

create policy "Users can insert their own tasks"
on public.tasks
for insert
with check (auth.uid() = user_id);

create policy "Users can update their own tasks"
on public.tasks
for update
using (auth.uid() = user_id)
with check (auth.uid() = user_id);

create policy "Users can delete their own tasks"
on public.tasks
for delete
using (auth.uid() = user_id);

alter publication supabase_realtime add table public.tasks;
```

5. Setup Instructions

- Install dependencies with pnpm install.
- Copy .env.example to .env and set VITE_SUPABASE_URL plus VITE_SUPABASE_ANON_KEY.
- Enable Anonymous sign-in in Supabase Auth.
- Run the SQL in supabase/schema.sql.

- Start locally with pnpm dev.
- Deploy the Vite app to Vercel, Netlify, or Cloudflare Pages with the same two environment variables.

6. Tradeoffs and Next Steps

- I prioritized a stable, reviewer-friendly core board over adding a larger optional backend surface.
- If I had more time, I would add assignees, task comments, and a dedicated activity log.
- I would also add integration tests around anonymous auth bootstrapping and drag-and-drop persistence.